

MessageFoundry User Guide

Early Access · MessageFoundry v0.3.2 · September 2026

MessageFoundry receives, routes, transforms, and validates healthcare messages. It uses HL7 v2.x by default and accepts other payload formats. Write routing and handling in Python and keep the code in version control. This guide covers installation, Connections, Routers, Handlers, and daily operation.

Contents

- What MessageFoundry is, and how to use this guide
- Installing and running the engine
- Quickstart: send your first message
- Authoring Connections
- Authoring Routers and Handlers
- Operating with the console and the VS Code extension
- Monitoring dispositions and troubleshooting
- Where to go next

What MessageFoundry is, and how to use this guide

The engine handles HL7 v2.x and other payloads, including JSON, XML/SOAP, X12 EDI, and database records. Routing and transforms use Python. Connection settings can also use TOML, edited by hand or through VS Code. The asyncio service runs in the background. The browser **web console** at `/ui` and VS Code extension connect through its localhost HTTP/WebSocket API.

The mental model in one read

Configuration links components by name. There is no bundled channel object. Use these four building blocks:

- **Connection** — an endpoint that **receives** (`inbound()`) or **sends** (`outbound()`) messages: MLLP, TCP, File, REST, SOAP, Database, SFTP/FTP. Every message in or out is counted and logged.
- **Router** (`@router`) — a pure Python function bound to **one** inbound Connection. It sees every received message and returns the **name(s)** of the Handler(s) to forward to (or none, to filter).
- **Handler** (`@handler`): a pure Python function that filters and transforms a message. It returns `Send(...)` objects for outbound Connections.
- **Message store** — durable persistence + the staged queue. SQLite (WAL) by default. PostgreSQL or SQL Server for production.

The loader resolves component names when configuration loads. An inbound names a Router. That Router selects Handlers. Each Handler uses `Send` to name an outbound. See [samples/config/IB_ACME_ADT.py](#) and [samples/config/adt.py](#) for runnable examples.

Messages move through three stored stages: **ingress**, **routed**, and **outbound**. Ingress commits the raw message before acknowledgment. Routed creates one row per selected Handler. Outbound creates one per destination. Separate asyncio workers drain each stage. The store records outcomes as `RECEIVED` → `ROUTED` / `UNROUTED` → `PROCESSED` / `FILTERED` / `NOT_DEPLOYED` / `ERROR`. Check outcomes and alerts to confirm delivery. See [ADR 0001](#) for at-least-once delivery and Handler purity rules.

Who this guide is for

Operators install, run, and monitor the engine. **Configuration authors** build Connections, Routers, and Handlers. For rollout guidance, read [EARLY-ADOPTER-GUIDE.md](#). For component concepts, read [MENTAL-MODEL.md](#).

Where the depth lives — the reference map:

When you want...	Read
The full architecture, modularity standard, and dependency rules	ARCHITECTURE.md (diagrams: architecture-diagram.md)
Connection types, settings, and the graph (incl. <code>connections.toml</code>)	CONNECTIONS.md , ADR 0007
Translation tables (code sets) — the grid editor + <code>codeset</code> CLI	CODESETS.md , ADR 0033
Service settings, environments, and <code>env()</code> values	CONFIGURATION.md
Running as a Windows service (NSSM)	SERVICE.md
Auth, RBAC, audit, and TLS	SECURITY.md , DEPLOYMENT.md
PHI handling and encryption-at-rest	PHI.md
The staged pipeline / reliability. Payload-agnostic ingress. The read-only <code>db_lookup</code> . X12	ADR 0001 , ADR 0004 , ADR 0010 , ADR 0012
What is built vs. Planned	FEATURE-MAP.md , README.md

A note on PHI before you run anything: this engine carries PHI, and a few commands (notably `dryrun` and `generate`) print **full message bodies** to stdout. Use **synthetic HL7 only** in examples, and never redirect that output to a committed file, ticket, or CI log. For realistic test data, use `python -m tee anonymize-captures` in the standalone tee relay. Its de-identification framework rejects unsafe output ([ADR 0030](#)). This provides test data from real traffic without manual PHI handling. See [PHI.md](#) for the hard rules.

How the rest of this guide is laid out

Follow these sections in order on your first run:

- 1. **Install** the engine and scaffold your config repo.

2. Send your **first message** end to end (e.g. `samples/messages/adt_a01.hl7` via `samples/send_mllp.py`) against the bundled sample config.
3. **Author Connections** (in Python or `connections.toml`).
4. **Author Routers and Handlers** to route and transform.
5. **Operate** via the browser web console (`/ui` , `messagefoundry-webconsole`) and the VS Code extension (`ide/`).
6. **Monitor and troubleshoot** with dispositions, alerts, dead-letter triage, and replay.

Work through them in order the first time. Afterward, jump to the task you need.

Installing and running the engine

This section uses a source checkout and the bundled `samples/config`. It also links to the pinned-wheel installation steps for deployments.

Two install styles exist. To **try MessageFoundry against the sample config** (this guide's running example), install from a checkout (`pip install -e .`). For deployment, install the pinned wheel. Create a configuration repository with `messagefoundry init`. Refer to `INSTALL-GUIDE.md`.

1. Check prerequisites

- **Python 3.14+** (64-bit). Everything else (the SQLite store, NSSM for the service) is auto-provisioned or bundled.
- **git** if you are cloning the checkout or standing up a config repo.
- **Administrator/elevation** only for the Windows service step (6).

Full hardware, OS, database, and port details: `SYSTEM-REQUIREMENTS.md`.

2. Install the engine

Create a virtual environment and install. For the **checkout / contributor** path (gives you `samples/config`, `samples/messages/`, and `samples/send_mllp.py`):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1      # Linux/macOS: . .venv/bin/activate
pip install -e .                  # core runtime + SQLite store
```

For a **deployment**, pin the published wheel instead (and verify its provenance — see `INSTALL-GUIDE.md`):

```
pip install "messagefoundry==0.3.2"
```

Add only the extras a host actually needs (each is opt-in and lazy-imported):

```

pip install -e packaging/messagefoundry-webconsole # the browser web console (/ui) – the operator UI (step 5)
pip install -e ".[harness]" # the standalone PySide6 test harness GUI
pip install -e ".[postgres]" # PostgreSQL store backend
pip install -e ".[sqlserver]" # SQL Server store backend – also needs the OS-level Microsoft ODBC Driver 18
pip install -e ".[sftp]" # SFTP transport for the REMOTEFILER connector
pip install -e ".[fhir]" # FHIR codec (R5/R4B/STU3) + FHIR() outbound (ADR 0022)
pip install -e ".[dicom]" # DICOM C-STORE SCP + codec – headers/SR only, no pixels (ADR 0025)
pip install -e ".[webauthn]" # browser WebAuthn passkeys for the /ui console (ADR 0068)
pip install -e ".[otel]" # OpenTelemetry/OTLP export seam (the /metrics endpoint itself needs no extra)

```

(For a deployment wheel, the same extras apply: `pip install "messagefoundry[harness]==0.3.2"`, and the web console installs as its own wheel `pip install "messagefoundry-webconsole==0.2.15"`.) SQLite is the zero-dependency default — you need no extra to run the sample config.

3. Run the engine headless (dev)

From the repo root, against the bundled sample config:

```
python -m messagefoundry serve --config samples/config --db ./messagefoundry.db --env dev
```

- `--config` points at the directory of Connection/Router/Handler modules (here `samples/config/`, which includes `IB_ACME_ADT.py` and its transform `adt.py`).
- `--db` is the SQLite message store path (created on first run).
- `--env` is required — `serve` refuses to start without it. The active environment is a free-form **name** (`dev` / `staging` / `prod`, or a custom name) that does two things: it selects the value file `environments/<env>.toml` that `env("...")` lookups resolve against, and it sets the instance's **PHI posture** (`data_class` / `production`). Built-in names carry a default posture. A custom name must declare it. See `CONFIGURATION.md`.

If the working directory differs from the repository root, set `--project-root <repo-root>`. This anchors value files for `env()` resolution. Refer to `INSTALL-GUIDE.md`.

Network / auth posture. The API binds `127.0.0.1:8765` and **requires authentication** by default. A non-loopback bind without TLS is refused at startup. Configure native TLS (or an upstream terminator) to expose it. Details: `SECURITY.md` and `DEPLOYMENT.md`.

Store encryption (PHI instances). If `[security].handles_real_patient_data = true`, `serve` requires a store encryption key. Without one, startup fails in every environment, including `dev` and `staging`. Mint one with `messagefoundry gen-key` (set it as `MEFOR_STORE_ENCRYPTION_KEY`), or on Windows DPAPI-protect it to a file with `messagefoundry protect-key --generate --out <file>` and point `[store].encryption_key_file` at it. The full key story is in `PHI.md`.

Confirm it is up:

```
curl http://127.0.0.1:8765/health
```

4. Scaffold your own config repo

Create a separate configuration repository for your own interfaces:

```
messagefoundry init ./my-config-repo  
cd my-config-repo
```

The command creates a starter feed in `config/` and a synthetic fixture in `messages/sets/`. It adds `environments/dev.toml`, `prod.toml`, and `messagefoundry.toml`. It also adds a pinned `requirements.txt` and continuous integration `check.yml`. Validate and run it:

```
pip install -r requirements.txt  
messagefoundry check --config config --messages messages/sets  
messagefoundry serve --config config --env dev
```

Put it under version control with **Set Up Version Control & Checks** in the IDE (or a plain `git init`). Select local or shared-remote repository storage during setup. Local storage suits development or a single machine without HA. Shared storage suits HA, team access, or off-machine backup. Change it any time with **Config Repo Storage Location**. Choosing local vs. Remote storage, and why secrets/PHI never land in the repo, are covered in `VERSION-CONTROL.md` (with the full deployment model in `INSTALL-GUIDE.md`).

5. Open the admin console (in a browser)

Install the `messagefoundry-webconsole` wheel alongside the engine to serve the browser console at `/ui`. It is **on by default** for loopback binds. Disable it with `[security].serve_web_console = false`. With the engine running, open:

```
http://127.0.0.1:8765/ui
```

The web console prompts for sign-in (authentication is on by default). Source: `packaging/messagefoundry-webconsole/`. (The former PySide6 desktop console was retired — BACKLOG #103. PySide6 now backs only the standalone test harness.)

6. Run as a Windows service (NSSM)

For Windows production deployment, use **NSSM** to run the engine as a service. NSSM starts it at boot and restarts it after a crash. It captures stdout/stderr in rotating logs. On stop, it sends Ctrl+C so connections drain.

Run the following command from an **elevated** PowerShell:

```
.\scripts\service\install-service.ps1 -Environment prod
```

`-Environment` is **required** (it becomes `serve --env`, just like step 3). Rerun the script to reconfigure the service. If NSSM is absent from `PATH`, it downloads a SHA-256-pinned copy. Defaults are service `MessageFoundry`, configuration `<repo>\samples\config`, store and logs `C:\ProgramData\MessageFoundry`, and bind `127.0.0.1:8765`. Override paths/port/account with flags, e.g.:

```
.\scripts\service\install-service.ps1 -Environment prod -Port 9000 `
  -Config D:\hl7\config -DataDir D:\MessageFoundry `
  -ServiceAccount "NT SERVICE\MessageFoundry" # least-privilege; auto-grants the needed ACLs
```

Manage and remove it:

```
nssm start MessageFoundry
nssm status MessageFoundry
nssm stop MessageFoundry
.\scripts\service\uninstall-service.ps1 # elevated; leaves logs + store in place
```

`SERVICE.md` covers accounts, configuration/log permissions, DPAPI key protection, updates, reinstallation, and troubleshooting.

A note on PHI-emitting commands

`messagefoundry dryrun` and `messagefoundry generate` print **full message bodies** to stdout/stderr (`dryrun` only with `--show-phi`. Redacted otherwise). Use synthetic HL7 only. Never use real PHI. Never redirect output to a committed file, ticket, or continuous integration log. See `PHI.md`.

Quickstart: send your first message

Use the shipped `sample configuration` to send a synthetic HL7 ADT message over MLLP and archive it to a file. No edits are required.

The `IB_Test_ADT` inbound in `samples/config/adt.py` listens on MLLP **port 2575**. Its Router sends `ADT` messages to the `archive` Handler. That Handler writes `./out/adt/{MSH-10}.hl7` through `FILE-OUT_Test_ADT`. Other sample feeds include ACME ADT on 2600, X12, and immunizations. This quickstart uses only `IB_Test_ADT`.

1. Start the engine

In one terminal, from the repo root, run the engine against the sample config. The active `--env` is **required** — use `dev` for local work:

```
python -m messagefoundry serve --config samples/config --db ./messagefoundry.db --env dev
```

The command loads configuration, opens or creates `./messagefoundry.db`, serves the localhost API, and starts the sample listeners. Startup logs the environment and security settings. Leave it running. The API defaults to `127.0.0.1`. See `INSTALL-GUIDE.md` for first-run sign-in and console setup.

2. Send the sample message

In a **second** terminal (also from the repo root), send the shipped synthetic ADT^A01 over MLLP with the helper in `samples/send_mllp.py`:

```
python samples/send_mllp.py samples/messages/adt_a01.hl7
```

The script defaults to `--host 127.0.0.1 --port 2575`, which matches `IB_Test_ADT`, so no flags are needed. To be explicit (or to target a different listener), pass them:

```
python samples/send_mllp.py samples/messages/adt_a01.hl7 --host 127.0.0.1 --port 2575
```

The file it sends, `samples/messages/adt_a01.hl7`, is synthetic — never substitute real PHI here.

3. What to expect

`send_mllp.py` reuses the engine's MLLP framing, waits for the framed ACK, and prints it:

```
--- ACK ---  
MSH|^~\&|...|...|...|ACK^A01|...|P|2.5.1  
MSA|AA|MSG00001  
...
```

`MSA|AA` means Application Accept: the engine received and stored the message. It does not confirm downstream delivery. See [ARCHITECTURE.md](#). The `ADT^A01` sample goes to the `archive` Handler. The Handler maps the sending facility only when a mapping exists. None exists for this sample, so MSH-4 stays unchanged. The file outbound writes to `./out/adt/`, and status moves `RECEIVED` → `ROUTED` → `PROCESSED`. Non-`ADT` messages become `UNROUTED`. An `ADT` event rejected by the Handler becomes `FILTERED`.

Confirm the delivered file landed (its name is the message's MSH-10 control ID, `MSG00001`):

```
ls ./out/adt/
```

4. Where to see it land

Two complementary ways to inspect what happened:

- **The web console Messages page.** Open `/ui` in a browser. Select Traffic → Messages. Find the message and inspect its result, original raw body, and per-destination delivery. The web console talks only to the engine's API — see [packaging/messagefoundry-webconsole/](#).
- **A dryrun preview (no engine needed).** To see exactly how the config *would* route and transform a message without sending it anywhere, run:

```
python -m messagefoundry dryrun --config samples/config --messages samples/messages/adt_a01.hl7 --inbound IB_Test_ADT
```

Dryrun prints the resolved inbound, disposition, selected handlers, and would-send payloads (`--inbound` names which inbound to evaluate — required here because the sample config wires several). **Its output is PHI-bearing** — message bodies are redacted by default and only included with `--show-phi`. The samples here are synthetic, but never pipe `dryrun` (or `generate`) output to a committed file or a CI log.

Now change what it does

Edit the Router and Handler in `samples/config/adt.py` to change behavior. See [Authoring Routers and Handlers](#). Add or retarget connections in `samples/config/connections.toml`, by hand or through the VS Code editor (ADR 0007). Follow the naming convention in [CONNECTIONS.md](#). Before committing, run `python -m messagefoundry check --config samples/config --messages samples/messages`.

Authoring Connections

A **Connection** is an endpoint that either *receives* messages (an **inbound** source) or *sends* them (an **outbound** destination). MLLP and File are the two most common transports, both shipped today (plus raw TCP, X12, REST, SOAP, Database, and SFTP/FTP — see the full catalog and per-setting reference in [CONNECTIONS.md](#)). Connections carry only *transport* config. Routing/transform *logic* lives in code-first Routers and Handlers (see [Authoring Routers and Handlers](#)).

Define connections in `.py` modules or `connections.toml`. Both load into the same registry and can coexist.

Name your connection

Use the convention `[TYPE]_[PARTNER]_[MESSAGE]`:

- **TYPE** — transport + direction code: `IB` / `OB` (inbound/outbound MLLP), `FILE-IN` / `FILE-OUT`, `TCP-IN` / `TCP-OUT`, `X12-IN` / `X12-OUT`, etc. (the full table is in [CONNECTIONS.md](#)).
- **PARTNER** — the system on the other end (`ACME`, `Epic`, `Test`).
- **MESSAGE** — the HL7 message code (`ADT`, `ORU`, `VXU`, ...) or `MIXED` / `ALL`.

Example: `IB_ACME_ADT` = inbound MLLP from ACME carrying ADT. Names are plain strings, so hyphens and mixed case (`FILE-OUT_Test_ADT`) are fine. Router/Handler names are *not* connections and do not follow this formula.

Author a code-first inbound and outbound

In a config module, call the `inbound()` / `outbound()` factories with a transport spec (`MLLP()`, `File()`, ...). Here is the shape, drawn from `samples/config/IB_ACME_ADT.py`:

```
from messagefoundry import MLLP, env, inbound, outbound

inbound("IB_ACME_ADT", MLLP(port=2600), router="acme_adt_router")
outbound("OB_ACME_ADT", MLLP(host=env("acme_adt_host"), port=env("acme_adt_port", cast=int)))
```

Key points the sample demonstrates:

- **Inbound MLLP takes only a port** — passing `host` is a wiring error. The listen interface is the service-level `[inbound].bind_host` (loopback in DEV, a specific NIC in PROD), an operator setting, not authored here.
- **Outbound MLLP needs a host and port**. Use `env("key")` for environment-specific peers or credentials. Values resolve from `environments/<env>.toml` and secrets from `MEFOR_VALUE_<KEY>`. The same module runs in each environment. A referenced-but-undefined value fails loud at load, never a silent blank host.
- For a **File** endpoint, use `File(directory="./out/adt")` (in) / `File(directory=..., filename="{MSH-10}.hl7")` (out). For non-HL7 bodies, set the inbound `content_type` to select `RawMessage` instead of HL7 parsing. Available types are `hl7v2` (default), `json`, `xml`, `text`, `x12`, `fhir`, `binary`, and `dicom`. `x12` rides any transport (see `samples/config/IB_PARTNER_X12.py`). `fhir` (ADR 0022) and `dicom` (ADR 0025) load their codecs on demand. The base64 `binary` path carries arbitrary bytes, including NUL (ADR 0028). `FHIR()` is **outbound-only** (a FHIR REST server facade is not shipped). `DICOM()` supports inbound C-STORE SCP and outbound C-STORE SCU / C-ECHO. The outbound sends to a downstream PACS. Set `host` and `called_ae_title` for that peer.
- Use `with_smart_backend(...)` for `FHIR()` or `Rest()` destinations on SMART-secured servers (for example, Epic or Oracle Health). It provides OAuth2 client-credentials and signed-JWT authentication (ADR 0024). Import it as `from messagefoundry.transports.smart import with_smart_backend` — it is not re-exported from the top-level package.

The complete per-connector settings (TLS, retry, DoS guards, ACK mode, `simulate`, etc.) are documented in `CONNECTIONS.md`. Each factory in `messagefoundry/config/wiring.py` **is the schema** for its transport.

Bind an inbound to its Router

An inbound names its Router with the `router=` keyword (`router="acme_adt_router"` above). The string must match a `@router` declared in some `.py` module loaded from the config dir. Names resolve **globally** across the directory, so the inbound and its router can live in separate files. An inbound with no matching router fails `messagefoundry check`. (The router and handler are authored in code — covered in [Authoring Routers and Handlers](#).)

Validate the wiring before running it:

```
messagefoundry check --config samples/config --messages samples/messages/adt_a01.hl7
```

Author a connection as data: `connections.toml`

A connection's transport config may instead live as **data** in an optional `connections.toml` next to the `.py` modules (ADR 0007). The loader merges these into the **same** registry the factories produce, so runtime, validation, and egress gating are identical. **Routing/transform logic stays code-first** — a data-authored inbound still binds a `router` declared in a `.py` module. Here is the shape, from `samples/config/connections.toml`:

```
[[inbound]]
name      = "IB_ACME_ADT_TCP"      # a second ACME intake, authored as data
transport = "mllp"
```

```
router    = "acme_adt_router"      # binds a router registered in a .py module
[inbound.settings]
port = 2700                        # inbound MLLP takes only a port
```

- The `transport` selects the same factory (`mllp` → `MLLP()`). TOML produces an identical specification with the same defaults and guards.
- **Secrets and per-environment peers use** `{ env = "key" }`, never an inline value (e.g. `host = { env = "acme_adt_host" }`, `port = { env = "acme_adt_port", cast = "int" }`). The file is repo-versionable and diffable.
- A name declared in **both** a `.py` module and `connections.toml` is a hard error (no silent shadowing).

Edit by hand or through the command-line tool, which the VS Code editor also uses. It preserves comments and formatting, and validates before saving:

```
messagefoundry connection list --config samples/config
messagefoundry connection upsert --config samples/config --data '{...}'
messagefoundry connection remove --config samples/config --name IB_ACME_ADT_TCP
```

`upsert` / `remove` validate the whole config dir (structure + connector build + the fail-closed `[egress]` allowlist) before persisting and roll back on failure.

Try it end-to-end

Run the engine against the dev environment (the active `--env` is required), then send a synthetic message at the inbound's port:

```
python -m messagefoundry serve --config samples/config --db ./messagefoundry.db --env dev
python samples/send_mllp.py samples/messages/adt_a01.hl7
```

Use only synthetic HL7 (as in `samples/messages/`) — never real PHI on a test feed.

Authoring Routers and Handlers

Register Python routing and transform functions with `@router` and `@handler`. Routers choose Handlers for each received message. Handlers filter, transform, and return `Send`s to outbound connections. Components link by name. See `samples/results_relay/results_relay.py` for an end-to-end template or `samples/config/adt.py` for a simple pair.

1. Write a Router (`@router`)

A Router returns Handler names. Return `[]` to select none. The engine records `UNROUTED`. Use tolerant field reads to make routing decisions.

From `samples/config/adt.py`:

```
@router("adt_router")
def route(msg):
    if msg["MSH-9.1"] != "ADT":
        return [] # not ADT - routed nowhere (logged UNROUTED)
    return ["archive"]
```

The Router name ("adt_router") is what an inbound Connection binds to: `inbound("IB_Test_ADT", MLLP(port=2575), router="adt_router")` . For the wiring conventions and per-connector settings, see [CONNECTIONS.md](#).

2. Write a Handler (@handler)

A Handler receives the message from a Router, then **filters** → **transforms** → **returns** `Send(s)`. Return `None` to filter the message out (logged `FILTERED`). Return one `Send` or a list to fan out to multiple outbound connections.

From `samples/config/adt.py`:

```
@handler("archive")
def archive(msg):
    if msg["MSH-9.2"] not in EVENT_LABELS:
        return None # only admit/register/update events are archived (others FILTERED)
    mnemonic = FACILITY_MNEMONICS.get(msg["MSH-4"])
    if mnemonic:
        msg["MSH-4"] = mnemonic
    return Send("FILE-OUT_Test_ADT", msg)
```

The `Send` target ("FILE-OUT_Test_ADT") names an `outbound(...)` Connection declared in the same config. To fan out, return a list — e.g. `samples/results_relay/results_relay.py` ends with `return [Send(OB_EHR, msg), Send(FILE_ARCHIVE, msg)]` .

3. The Message operations you'll use

Routers and Handlers work against the mutable HL7 `Message` in `messagefoundry/parsing/message.py` — never string-slice raw HL7. Read/mutate through `Message` and re-encode. The methods you will reach for (see the docstrings in that file for full signatures):

- **Peek a field** — `msg["PID-3"]` / `msg.field("OBX-3.1", occurrence=i)` . Convenience properties `msg.message_code` (MSH-9.1), `msg.trigger_event` (MSH-9.2), `msg.control_id` (MSH-10).
- **Iterate repetitions / segments** — `msg.repetitions("PID-3")` walks a ~-list. `msg.count_segments("OBX")` plus `field(occurrence=...)` walks repeating segments.
- **Mutate** — `msg["MSH-4"] = value` / `msg.set(path, value, occurrence=..., repetition=...)` . Rebuild a repeating block with `msg.delete_segments("OBX")` + `msg.add_segment(line, index=...)` + `msg.add_repetition(...)` .
- **Read MSH separators** — never hardcode `|^~\&` . The repeating-segment rebuild in `samples/results_relay/results_relay.py` reads them from MSH-1/MSH-2 (its `_separators` helper) before joining components.

- **Re-encode** — `msg.encode()` (or just pass `msg` to a `Send`, which encodes for you).

A non-HL7 inbound (`content_type` other than `hl7v2`) supplies a `RawMessage`. Read `.raw`, `.text`, `.json()`, or `.xml()`. The defusedxml-based XML accessor rejects DOCTYPE, external entities, and billion-laughs payloads. Use `Send` with the resulting string. For cross-field business-rule checks beyond what schema validation catches, compose the primitives in `parsing/consistency.py`, as `samples/consistency/validated_adt.py` does (raise `ConsistencyError` → dead-letter, or `return None` → filter). The three validation tiers are laid out in `HL7-VALIDATION.md`.

4. Translation tables (code sets)

Routers and Handlers can translate facility codes, order codes, or bed locations for a destination. The earlier `FACILITY_MNEMONICS` example maps facility codes. A translation table, also called a code set, stores these mappings in `codesets/<name>.csv`. The file stem supplies the table name. The graph loads the table and reloads it on promotion. Lookups are pure, so the staged pipeline can repeat them safely.

`codesets/facility_mnemonics.csv`:

```
sending_facility,mnemonic
ACME_MAIN,ACMS
ACME_WEST,ACMW
```

Read it with `code_set("name")`, which returns a frozen, read-only mapping. Capture it once at a module's top level (or call it inline in a handler — both resolve):

```
from messagefoundry import code_set, handler, Send

FACILITIES = code_set("facility_mnemonics")    # loads with the graph; reloads on promote

@handler("archive")
def archive(msg):
    src = msg["MSH-4"]
    msg["MSH-4"] = FACILITIES.get(src, src)    # translate; pass through unchanged if unmapped
    return Send("FILE-OUT_Test_ADT", msg)
```

- **Missing key — you choose the behavior.** `code_set(...).get(key, default)` returns `default` on a miss (pass-through as above, or `""` to blank it). The subscript `code_set(...)[key]` **raises** on a miss, sending that message to its `ERROR` /dead-letter disposition (the strict "never deliver an unmapped value" path).
- **Single vs. Multi-column.** One value column → the value is a scalar string. Two or more → it is a `{header: cell}` dict (`code_set("x")["k"]["mnemonic"]`). Keys are exact-match, **case-sensitive** strings.

Edit tables manually or through the VS Code Translation Tables grid (New / Edit Translation Table). The grid uses the offline `messagefoundry codeset` validation tool:

```
messagefoundry codeset list --config samples/config
messagefoundry codeset upsert --config samples/config --data '{...}' # validate → atomic write
messagefoundry codeset rename --config samples/config --name old --to new
```

```
messagefoundry codeset remove --config samples/config --name old
```

The engine loader validates each table before an atomic save. It rejects duplicate keys and malformed files. A failed save restores the previous table. Promote the change with `POST /config/reload`.

After a rename or removal, run `messagefoundry check`. A Handler resolves `code_set("old_name")` at runtime. Plain `validate` cannot detect a missing runtime reference, but the dry-run in `check` can.

Refer to [CODESETS.md](#), [CONFIGURATION.md](#), and [ADR 0033](#). For external lookup files, see reference sets in [ADR 0006](#). For live database reads, see `db_lookup` below.

5. Purity rule (don't break this)

Routers and Handlers **must be pure**: they return messages without external side effects. The engine can rerun transforms after crashes, so repeated execution must produce identical output. Put network, file, and database writes in outbound Connections.

Two read-only exceptions are available inside live Handlers: `db_lookup(connection, statement, params)` and `fhir_lookup(connection, query)`. FHIR supports GET-only reads by ID (`"Patient/123"`) and searches (`"Patient?identifier=MRN|123"`). Results may change on rerun. This is accepted by design. Both run outside the event loop. `db_lookup` uses `[egress].allowed_db`. `fhir_lookup` uses `[egress].allowed_http`. Both reject unlisted destinations. They raise errors in Routers or dry-runs. See [ADR 0010](#) and [ADR 0043](#).

6. The authoring dev loop

Run these from your config-repo root as you write — they touch no network and start no server.

```
# Static check: structural problems in the config dir
python -m messagefoundry validate --config samples/config

# See the wired Connection → Router → Handler → Connection graph
python -m messagefoundry graph --config samples/config

# Run real messages through Routers/Handlers WITHOUT sending — shows the disposition,
# selected handlers, and would-send payloads. Bodies are redacted unless you pass --show-phi.
# (--inbound selects which inbound to evaluate; it's required when the config wires more than one.)
python -m messagefoundry dryrun --config samples/config --messages samples/messages/adt_a01.hl7 --inbound
IB_Test_ADT

# The commit/CI gate: validate + dryrun (+ advisory ruff/mypy). Exit 0 only if every required check
passes.
python -m messagefoundry check --config samples/config --messages samples/messages
```

`dryrun --show-phi` prints full raw and intended-output message bodies to stdout. The default output redacts PHI. Use synthetic HL7 only. Do not save output in a committed file, ticket, or continuous integration log.

Add `check` to the pre-commit hook and continuous integration workflow. Refer to [CONNECTIONS.md](#) for connection settings and [HL7-VALIDATION.md](#) for validation tiers.

Operating with the console and the VS Code extension

The **web console** at `/ui` operates a running engine: manage connections, inspect messages, monitor health, and manage users. The **VS Code extension** builds, tests, and promotes configuration. Both use the engine API and never access the database directly. The former PySide6 desktop console is retired (BACKLOG #103). PySide6 now serves only the standalone test harness.

Opening and signing in to the console

The console is served by the engine itself, so start the engine first (note the active `--env` is required):

```
python -m messagefoundry serve --config samples/config --db ./messagefoundry.db --env dev
```

Then open the web console in a browser (the engine serves it at `/ui` by default, with the `messagefoundry-webconsole` distribution installed):

```
http://127.0.0.1:8765/ui
```

When the engine requires authentication (the default), a **Sign in** form appears first:

1. Enter your **username** and **password**, and pick a **Provider** — *Local* always. *Active Directory* appears only if the engine advertises AD.
2. If your account uses TOTP, enter the 6-digit authenticator code or a single-use recovery code. The console checks it before it opens a page. An account with a **passkey** enrolled gets a *Use passkey* button here instead (ADR 0068. It needs the `[webauthn]` extra and a configured `public_origin`, and says so plainly when either is missing).
3. On a forced password change, the console chains a change-password step (local accounts only — AD passwords are changed in Active Directory).

After sign-in, the engine uses an `HttpOnly mf_session` cookie. Further sign-in is necessary only after the session expires or the engine revokes it.

Open **Account** → **My account** to change a password, manage MFA, manage passkeys, or view and revoke active sessions. TOTP enrollment shows a setup key and `otpauth://` URL for manual entry, then one-time recovery codes. The navigation menu contains **Sign out**.

Role-based access control determines which pages and actions you can use. Refer to [SECURITY.md](#).

A tour of the console pages

The top navigation menu has these groups:

- **Traffic:** Connections, Messages, Dead letters, Events.
- **Monitoring:** Status, Alerts, Flow & trends, Audit, Uploaded logs.
- **Admin:** Users, Configuration.
- **Account:** My account, My security events.

Menus open on hover or keyboard focus. All users see every menu item, but the server rejects pages without permission.

The right side shows an alert bell, a health indicator, and **Sign out**. The indicators refresh about every 15 seconds on every page. Green means healthy, orange means degraded, and blinking red means the engine or database stopped.

- **Connections** shows one row per inbound or outbound endpoint. Select rows, then choose **Start**, **Stop**, **Restart**, or **Reset stats**. For a stopped and quiesced outbound, **Purge top** and **Purge all** cancel queued deliveries permanently. Purge requires re-authentication and may require a second approver. The connection name opens its **Messages** search. The ⓘ link opens read-only details. Columns are Flag, Connection, Dir, Status, In, Out, Queued, Errors, Alerts, and Idle. A startup build or bind failure shows **failed** without stopping the engine (ADR 0031). See [troubleshooting](#).
- **Messages** filters by connection, status, type, control ID, and receipt time. Open a message to see metadata, the original **Raw message**, **Deliveries**, and **Events**. Delivery details include status, attempts, and last error. **Parse tree** → uses the pure **parsing** library to open the HL7 view without rendering message bodies as markup. **Replay** sends the message again. **Edit & resubmit** creates a copy. The server audits raw-body access with your username. **Content search** searches bodies by field path or substring. This bulk PHI decryption requires re-authentication and an audit record.
- **Status** — read-only health: engine (version, uptime, PID, inbounds running/total, endpoints in+out, engine-wide msg/s), store (path, size, free disk, journal mode, message/event/audit row counts), the effective security posture, and the active-passive **Cluster** roster + DR state. **Run integrity check** runs **PRAGMA quick_check** on demand, and **Reset statistics** zeroes the counters. When `[service].report_status` is on, Hosting service shows the NSSM service state. It has no start/stop controls. Manage the service itself per [SERVICE.md](#).
- **Users** requires `users:read` for viewing and `users:manage` for changes. Use **+ New user** to create a user. The user page has Profile, Roles, Channel scope, and account actions. Actions include *Reset password*, *Reset MFA*, *Sign out all sessions*, and *Delete user*. **Roles** defines custom roles from the permission catalog (ADR 0045). **AD group mappings** links directory groups. Forms that contain bodies require a fresh re-authentication window. The server audits each operation. See [SECURITY.md](#).
- **Alerts** — the engine's **active alert instances** (open and acknowledged) plus the loaded `[alerts]` rules (ADR 0044 refining ADR 0014). Each instance records severity, status, event type, connection, occurrence count, first/last seen, reason, and the user who acknowledged it. Actions are Ack, Resolve, windowed Suspend, and Resume. Rule *editing* stays config-file driven. The rule list is shown read-only (event type, connection, min depth, min age, severity, transports, cooldown — transports reported present-or-not, secrets omitted). The notifications themselves still fan out through the engine's AlertSink (see [Monitoring dispositions and troubleshooting](#)).

The **Dead letters** page lists failed deliveries, newest first. Each row represents a message and destination that exhausted retries. Columns are *Failed / Channel / Destination / Type / Attempts / Last error / Message*.

The page offers **Replay all dead** — `<connection>`, **Replay** `<connection>` → `<destination>`, and replay across all connections. Viewing requires `messages:read`. Replay requires `messages:replay` and re-authentication. A second approver may also be necessary.

The **Message** link opens the audited detail page. Its **Deliveries** section shows the error and offers per-message **Replay**. Refer to [EARLY-ADOPTER-GUIDE.md](#) for recovery instructions.

The VS Code extension (for config authors)

The VS Code extension runs the `messagefoundry` command-line tool to author and test configuration. It can start, stop, and restart a local engine and show its status. Use the web console to monitor traffic. The extension includes:

- Setup tools appear on Home. New Route Wizard generates Inbound → Router → Handler → Outbound as one module. New Connection generates a `[TYPE]_[PARTNER]_[MESSAGE]` module, such as `IB_ACME_ADT`. Other tools create Routers and Handlers. Generate Samples uses `messagefoundry generate` for conformant synthetic data without PHI. Set Up Version Control & Checks adds Git and a `messagefoundry check` pre-commit hook.
- **Validate + graph** — *Validate on save* surfaces problems in the Problems panel. The Components view shows `messagefoundry graph` with convention names. Use Filter and Group to select the view. The row's  button opens `MLLP()` or `File()` settings in code.
- The Translation Tables grid creates, edits, renames, and removes translation tables, also called code sets. It shells the `messagefoundry codeset` CLI (validate-on-save, atomic write) and offers **Promote** to apply. See [CODESETS.md](#).
- **Test Bench** loads `.hl7` files and splits multiple messages on `MSH`. It runs the configuration without delivery and shows each message's result. **Before/After** shows the changes. **Debug** steps through code with `debugpy`. The load dialog uses `messagefoundry.messageSetsDir`, which defaults to `samples/messages`.
- **Stage → Promote** applies local configuration to a running engine. First, validate the configuration. Select the target environment. Run `POST /config/reload {dry_run:true}` against its `env()` values. Confirm the change. The engine then swaps the live graph atomically. The extension requires sign-in and stores the token in VS Code SecretStorage.
- AI assist uses the provider-agnostic `@messagefoundry` chat participant (`/explain`, `/transform`, `/router`, `/review`, `/migrate`, `/test`). It sends code and the configuration graph, without message bodies or PHI.

Full feature and settings reference: [ide/README.md](#).

PHI: `messagefoundry generate` and `dryrun`, used by the Test Bench, can print full message bodies. Use only synthetic HL7, such as `samples/messages/adt_a01.hl7`. Never redirect output to a committed file, ticket, or continuous integration log.

Monitoring dispositions and troubleshooting

The engine stores and counts each received message before acknowledging it. It records the message's outcome as processing advances. Use those outcomes to monitor delivery and recover failures. See [ARCHITECTURE.md](#) and [ADR 0001](#) for the delivery model.

What each disposition means

Messages move through the **staged pipeline**: `ingress -> routed -> outbound`. The store finalizer sets the final outcome only after every Handler's work resolves. One completed Handler cannot finalize a message while another remains active.

Status	What it means to you
RECEIVED	The engine stored and counted the message at ingress, then sent an ACK. Routing has not started.
ROUTED	The Router selected at least one Handler. Transform and delivery work remains.
UNROUTED	The Router selected no Handler. The engine records and keeps the message without delivery. This is not an error. Check the Router if you expected delivery.
FILTERED	Every Handler completed without delivery. Filter rules can intentionally cause this result. Check unexpected results against the Handler filters.
NOT_DEPLOYED	Every selected destination has <code>deployed = false</code> , so the engine queues no delivery (ADR 0111). The result is not an error or <code>FILTERED</code> . Each skipped destination produces a <code>not_deployed</code> event, even with <code>[diagnostics].message_events = "off"</code> . If another deployed destination receives the message, the final status is <code>PROCESSED</code> . Skipped destinations remain in the event record. To enable a destination, set <code>deployed = true</code> . Supply its <code>env()</code> values. Then reload configuration. Console start/restart returns <code>409</code> because deployment requires a configuration change. See CONNECTIONS.md .
PROCESSED	Every selected Handler completed the transform and all destinations received delivery.
ERROR / dead-letter	A stage failed. Decode, parse, or strict-validation failures occur before ingress and return a NAK. Later failures record <code>ERROR</code> and an <code>AlertSink</code> event after acknowledgment. A delivery that exhausts retries becomes a dead letter for inspection and replay.

The key operator shift under the staged pipeline: **an AA ACK means "received and persisted," not "delivered."** A post-ingress failure is a disposition + alert, not a NAK.

Where to watch dispositions

- **Web console → Messages.** Use the **status** filter to select a result, such as `unrouted` or `error`. Open a message to inspect its raw body, parse tree, deliveries, and audit trail. A connection name on the Connections page opens Messages with that connection selected.
- **Web console -> Connections.** The dashboard shows connection status and per-connection counts. The errored column shows error counts for each feed.

- **API.** `GET /messages?status=error` (and `&channel_id=`, `&message_type=`) is the filter the console uses. `GET /stats` returns outbox-by-status + in-pipeline depth. `GET /status` returns engine uptime, running/stopped channel counts, and DB size/free-disk. `GET /metrics` provides Prometheus data without PHI on a base installation. It includes aggregate counts and latency by connection and status. The `[otel]` extra adds optional OpenTelemetry/OTLP export. All require the `monitoring:read` (stats/status/metrics) or `messages:read` (the Messages browser) permission — see [SECURITY.md](#).

The ERROR / dead-letter path: inspect and replay

A delivery dead-letters when its retries are exhausted. Retry behavior is per-outbound (defaults in `[delivery]` — see [CONFIGURATION.md](#)):

- `retry_max_attempts` **unset = retry forever** (the conservative default. Under FIFO the failing head blocks its lane until it succeeds or is purged). Set a finite value to opt into retry-then-dead-letter.
- A partner **AR reject fails fast** (no retry). An **AE NAK / transient transport failure is retried** with backoff.

To recover:

1. **Find the dead-letters.** Console: open the **Dead letters** page (or the message itself from **Messages**) and read its delivery row's **Last error**. API: `GET /dead-letters` (optionally `?channel_id=&destination_name=`) lists dead deliveries newest-first. Each row carries `last_error`.
2. **Fix the cause** (the downstream endpoint, the transform, the config).
3. **Replay.** `POST /dead-letters/replay` re-queues the dead deliveries (optionally scoped by `channel_id` / `destination_name`). Each affected message reverts from `error` to `received` and re-drains. Already-delivered rows are left alone. Replay requires `messages:replay` permission and re-authentication. If `[approvals]` is configured, a second approver may also be required. (In the console, the **Dead letters** page lists these and offers the per-connection, per-destination, and replay-everything buttons directly. This API path is the equivalent for scripting and automation.)

Replaying re-transmits real message bodies — it is audited per acting user. Treat it like any PHI action ([PHI.md](#)).

Alerting: fire-and-forward notifications

The engine raises operational alert events — `connection_stopped` (a lane halted by the `stop` internal-error policy), `queue_buildup` (a backlog past its depth/age threshold), `storage_threshold` (the store grew past `[retention].max_db_mb`), and `cert_expiry` (a monitored TLS certificate nearing expiry) — through an `AlertSink`. With no `[alerts]` transport configured these are just logged at `WARNING`. Configure a transport and they fan out to it ([alert_sinks.py](#)).

Configure alerts in the `[alerts]` section ([CONFIGURATION.md](#)):

```
[alerts]
webhook_url = "https://hooks.example.com/mf"    # POSTs each event as JSON (Slack/Teams/PagerDuty)
email_smtp_host = "smtp.example.com"
email_from = "messagefoundry@example.com"
```

```
email_to = ["oncall@example.com"]
# Optional per-event routing/severity/suppression rules (first match wins) — ADR 0014:
[[alerts.rules]]
event_type = "connection_stopped"
severity = "critical"
transports = ["webhook"]
```

The SMTP password is a secret — supply it via `MEFOR_ALERTS_EMAIL_PASSWORD`, never the file. Per-event severity, transport routing, thresholds, suppression, and cooldown are tuned with ordered `[[alerts.rules]]` tables (ADR 0014). An event matching no rule notifies every configured transport at `warning`.

The engine attempts webhook/email delivery but keeps no notification send log. Check delivery at the webhook or email target. It separately stores **alert state** per `(event type, connection)`. Query `GET /alerts/active` and act through `POST /alerts/{id}/ack`, `/resolve`, `/suspend`, or `/resume`. The console uses these endpoints (ADR 0044). `GET /alerts/rules` shows loaded rules and transport configuration, with secrets and recipients omitted. Alert instances and payloads contain connection names and queue details, without message content or PHI.

Common problems

- **Sender got a NAK (AE/AR).** The listener rejects decode, parse, and strict-validation failures synchronously. It records `ERROR` before ingress, so the message never enters the pipeline. Fix the inbound HL7 (or relax `validation.strict` on that connection). Treat the message body as untrusted data, not a malformed instruction.
- **Sender got AA but nothing was delivered.** Expected under ACK-on-receipt: routing/transform/delivery failures happen *after* the ACK. Look at the message's disposition (`UNROUTED` / `FILTERED` / `NOT_DEPLOYED` / `ERROR`) and the AlertSink — **not** the ACK — for the outcome.
- **A lane stopped processing.** A `connection_stopped` alert means an outbound's worker halted on an internal/code error (`internal_error = stop`). The messages are preserved for replay. Fix the cause, then reload/restart the connection.
- **A connection shows failed.** A startup build or bind failure affects that connection only. Other connections continue (ADR 0031). Correct the settings or port conflict. For an inbound, use `POST /connections/{name}/start`. For a failed outbound, reload the configuration. Reload rejects an invalid configuration as a whole.
- **Backlog growing.** A `queue_buildup` alert usually means a retry-forever head is blocking its FIFO lane, or the downstream is down. Check the destination, then inspect/purge or replay the blocking row.
- **Console cannot reach the engine.** The API binds `127.0.0.1:8765` by default and requires auth. Confirm that the engine is running (`python -m messagefoundry serve --config samples/config --db ./messagefoundry.db --env dev`). Check that `messagefoundry-webconsole` is installed. Check that `[security].serve_web_console` enables the console. Open that host and port's `/ui` page.
- **Low disk / store growing.** `GET /status` reports DB size and free disk. A `storage_threshold` alert fires past `[retention].max_db_mb`. Configure `[retention]` (CONFIGURATION.md). Purges clear PHI bodies but retain message, result, and audit records. The row is kept. Its PHI columns — operator-attached `metadata` included — are blanked.

Where to go next

- **Concepts in depth** — [MENTAL-MODEL.md](#), [ARCHITECTURE.md](#) (diagrams: [architecture-diagram.md](#))
- **Narrative onboarding, install to production** — [EARLY-ADOPTER-GUIDE.md](#), [INSTALL-GUIDE.md](#), [SYSTEM-REQUIREMENTS.md](#)
- **Connections reference** — [CONNECTIONS.md](#), [ADR 0007](#) ([connections.toml](#))
- **Translation tables (code sets)** — [CODESETS.md](#), [ADR 0033](#)
- **Service settings & environments** — [CONFIGURATION.md](#)
- **Validation tiers** — [HL7-VALIDATION.md](#)
- **Run as a service** — [SERVICE.md](#)
- **Security, RBAC, TLS** — [SECURITY.md](#), [DEPLOYMENT.md](#)
- **PHI handling & encryption-at-rest** — [PHI.md](#)
- **VS Code extension** — [ide/README.md](#)
- **What is built vs. Planned** — [FEATURE-MAP.md](#), [README.md](#)
- **Design records** — [ADR 0001](#) (staged pipeline), [ADR 0004](#) (payload-agnostic ingress), [ADR 0010](#) ([db_lookup](#)), [ADR 0012](#) (X12), [ADR 0014](#) (alerting rules)

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.